

23/7/2026

L & T edutech

Theory Question Paper - Set B

Name : Richard Raju
 Roll Number : 2460432
 Class : 5 BTCS 'B'
 Date : 23/07/2026

Q.1. Write a program to check whether a given year is leap year using if-else.

Ans:

```

let year = 2024;
if((year % 4 === 0 && year % 100 !== 0) || year % 400 === 0) {
  console.log(year + "is a leap year.");
}
else {
  console.log(year + "is not a leap year.");
}
  
```

Q.2. The loop that checks its condition before executing its body is called a _____ loop.

Ans: The loop that checks its condition before executing its body is called a while loop.

Q.3. Write a program to print numbers from 1 to 20 skipping multiples of 3 using continue.

Ans:

```
for (let i = 1; i <= 20; i++)  
  if (i % 3 === 0)  
  {  
    continue;  
  }  
  console.log(i);  
}
```

Q.4. The method used to add a new element to the beginning of an array is _____

Ans: The method used to add a new element to the beginning of an array is unshift()

Q.5. Write a program to create an object book with properties title, author and price then print each property.

Ans:-

```
let book = {  
  title: "The Alchemist", author: "Paulo  
  Coelho", price: 499  
};  
console.log("Title:", book.title);  
console.log("Author:", book.author);  
console.log("Price:", book.price);
```

CIA - 1 EXAMINATION SUBMISSION

Node & Express JS — Practical Set-2

Name	Richard Raju
Date	23/07/2026
Semester	5th Semester
Practical Set	Set-2, Hospital Patient System — Tasks 1 to 10

Practical Answers - Hospital Patient System

Task 1: Hospital Patient System — Welcome Message

Question: Write a program that prints a welcome message for the "Hospital Patient System" to the console.

Answer (Node.js):

```
// Task 1 - Welcome Message
```

```
console.log("Welcome to the Hospital Patient System!");
```

```
// Expected Output:
```

```
// Welcome to the Hospital Patient System! Actual Output (Node.js terminal):
```

Actual Output (Node.js terminal):

```
PS D:\Set2> node task1.js
Welcome to the Hospital Patient System
```

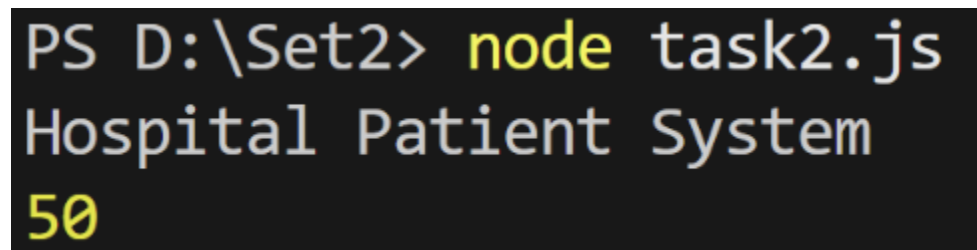
Task 2: Hospital Patient System — Declare and Update Variables

Question: Declare variables to store the system name and the total number of patients, then print both.

Answer (Node.js):

```
// Task 2 - Declare and Update Variables  
  
let systemName = "Hospital Patient System";  
  
let totalPatients = 50;  
  
console.log(systemName);  
  
console.log(totalPatients);  
  
// Expected Output:  
  
// System Name: Hospital Patient System  
  
// Total Patients: 50
```

Actual Output (Node.js terminal):

A screenshot of a Node.js terminal window with a black background. The prompt 'PS D:\Set2>' is followed by the command 'node task2.js' in yellow. The output consists of two lines: 'Hospital Patient System' and '50', both in yellow.

```
PS D:\Set2> node task2.js  
Hospital Patient System  
50
```

Task 3: Hospital Patient System — Identify Datatypes

Question: Declare variables of type Number, String, and Boolean related to a patient, and print their datatypes using `typeof`.

Answer (Node.js):

```
// Task 3 - Identify Datatypes
```

```
let age = 25;  
let name = "Rahul";  
let admitted = true;  
console.log(typeof age);  
console.log(typeof name);  
console.log(typeof admitted);
```

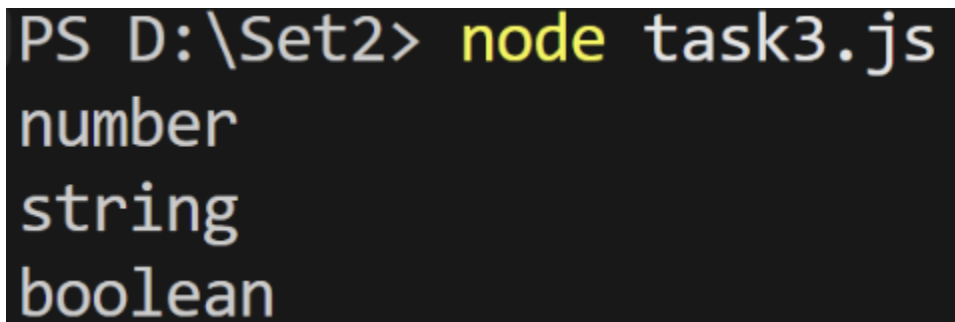
// Expected Output:

// number

// string

// boolean

Actual Output (Node.js terminal):

A screenshot of a Node.js terminal window with a black background and white text. The prompt 'PS D:\Set2>' is followed by the command 'node task3.js' in yellow. The output consists of three lines: 'number', 'string', and 'boolean', each on a new line.

```
PS D:\Set2> node task3.js  
number  
string  
boolean
```

Task 4: Hospital Patient System — Perform a Calculation

Question: Write a program to calculate the age difference between two patients using arithmetic operators, and print the result.

Answer (Node.js):

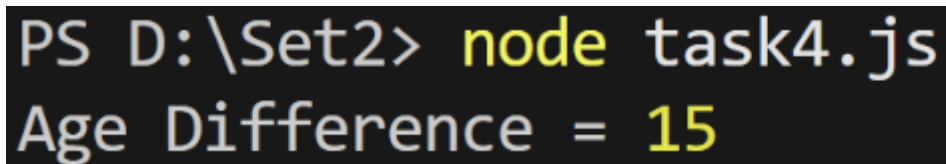
```
// Task 4 - Perform a Calculation
```

```
let age1 = 45;
let age2 = 30;
let difference = age1 - age2;
console.log("Age Difference =", difference);
```

// Expected Output:

// Difference: 15

Actual Output (Node.js terminal):



```
PS D:\Set2> node task4.js
Age Difference = 15
```

Task 5: Hospital Patient System — Classify a Record

Question: Write a program to check whether a given patient's `age` meets a specific condition using if-else.

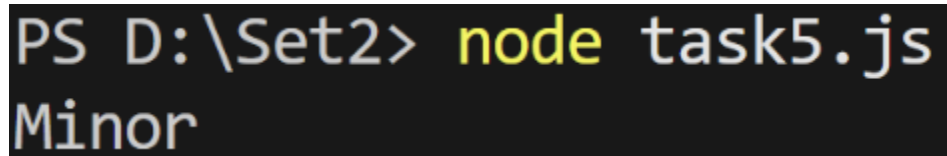
Answer (Node.js):

```
// Task 5 - Classify a Record
let age = 17;
if (age >= 18) {
  console.log("Adult");
} else {
  console.log("Minor");
}
```

//Expected Output:

Minor

Actual Output (Node.js terminal):

A screenshot of a Node.js terminal window. The prompt is 'PS D:\Set2>' and the command entered is 'node task5.js'. The output displayed is 'Minor'.

Task 6: Hospital Patient System — Iterate Over Records

Question: Create an array of at least 4 patient objects and use a loop to print each one's details.

Answer (Node.js):

```
// Task 6 - Iterate Over Records
```

```
let patients = [  
  { name: "Rahul", age: 25 },  
  { name: "Anu", age: 30 },  
  { name: "Ravi", age: 18 },  
  { name: "Neha", age: 40 }  
];  
for (let patient of patients)  
{  
  console.log(patient.name, patient.age);  
}
```

//Expected Output:

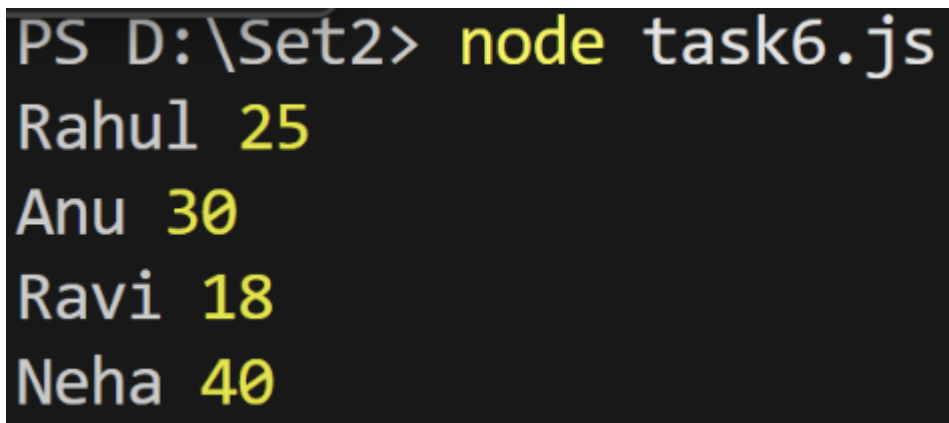
// Name: Rahul Age: 25

// Name: Anu Age: 30

// Name: Ravi Age: 18

// Name: Neha Age: 40

Actual Output (Node.js terminal):

A screenshot of a Node.js terminal window with a black background and yellow text. The prompt is 'PS D:\Set2>' followed by the command 'node task6.js'. The output consists of five lines: 'Rahul 25', 'Anu 30', 'Ravi 18', and 'Neha 40'.

```
PS D:\Set2> node task6.js
Rahul 25
Anu 30
Ravi 18
Neha 40
```

Task 7: Hospital Patient System — Skip or Stop Early

Question: Using the array of patients, use `break` or `continue` while looping to skip or stop at a specific condition on `age`.

Answer (Node.js):

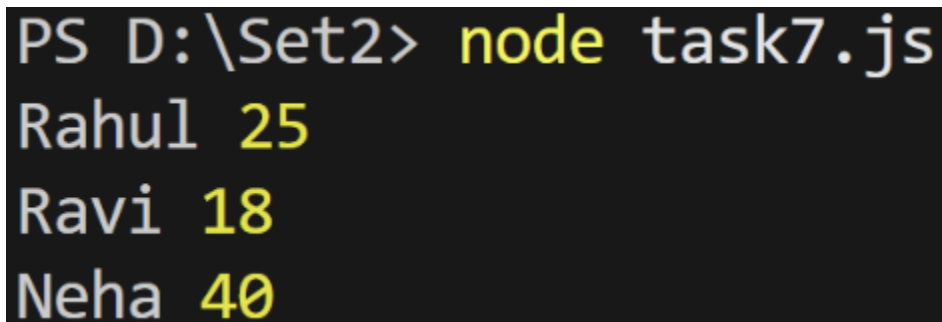
// Task 7 - Skip or Stop Early

```
let patients = [
  { name: "Rahul", age: 25 },
  { name: "Anu", age: 15 },
```



```
{ name: "Ravi", age: 18 },  
  { name: "Neha", age: 40 }  
];  
for (let patient of patients) {  
  if (patient.age < 18)  
    continue;  
  console.log(patient.name, patient.age);  
}  
// Expected Output:  
// Rahul - 25  
// Ravi - 40  
// Neha - 60
```

Actual Output (Node.js terminal):

A screenshot of a Node.js terminal window with a black background. The prompt is 'PS D:\Set2>'. The command 'node task7.js' has been executed, resulting in three lines of output: 'Rahul 25', 'Ravi 18', and 'Neha 40'. The text is displayed in a monospaced font with yellow and white colors.

```
PS D:\Set2> node task7.js  
Rahul 25  
Ravi 18  
Neha 40
```

Task 8: Hospital Patient System — Add & Find

Question: Use `push()` to add a new patient to the array. Write a loop to find the patient with the highest `age`.

Answer (Node.js):

```
// Task 8 - Add & Find

let patients =

[
  { name: "Rahul", age: 25 },
  { name: "Anu", age: 30 },
  { name: "Ravi", age: 18 }
];

patients.push({ name: "Neha", age: 40 });

let oldest = patients[0];

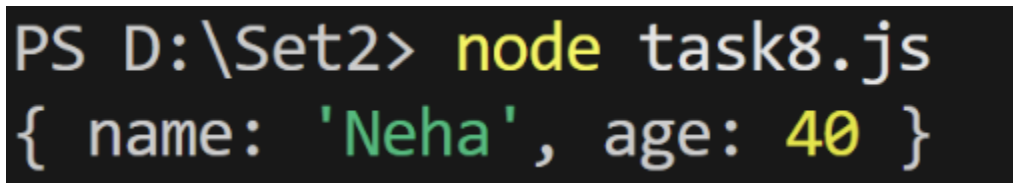
for (let patient of patients) {
  if (patient.age > oldest.age)
    oldest = patient;}

console.log(oldest);
```

// Expected Output:

```
// { name: 'Neha', age: 40 }
```

Actual Output (Node.js terminal):



```
PS D:\Set2> node task8.js
{ name: 'Neha', age: 40 }
```

Task 9: Hospital Patient System — Remove & Sort

Question: Use `pop()` or `shift()` to remove an entry from the array. Use the array's `sort()` method to sort patients by `age`.

Answer (Node.js):

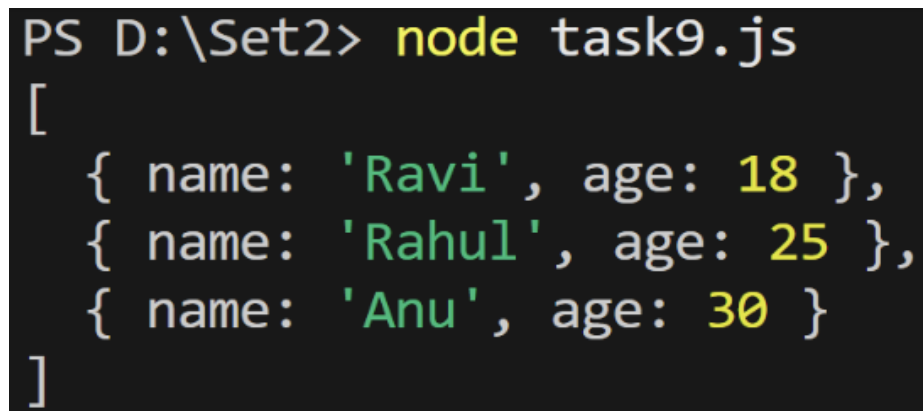
// Task 9 - Remove & Sort

```
let patients = [  
  { name: "Rahul", age: 25 },  
  { name: "Anu", age: 30 },  
  { name: "Ravi", age: 18 },  
  { name: "Neha", age: 40 }];  
  
patients.pop();  
  
patients.sort((a, b) => a.age - b.age);  
  
console.log(patients);
```

// Expected Output:

```
// [  
//   { name: 'Kiran', age: 17 },  
//   { name: 'Rahul', age: 25 },  
//   { name: 'Anita', age: 40 }  
// ]
```

Actual Output (Node.js terminal):



```
PS D:\Set2> node task9.js  
[  
  { name: 'Ravi', age: 18 },  
  { name: 'Rahul', age: 25 },  
  { name: 'Anu', age: 30 }  
]
```

Task 10: Hospital Patient System — Create and Print an Object

Question: Create a `patient` object with at least 5 properties and print each property individually.

Answer (Node.js):

```
// Task 10 - Create and Print an Object
```

```
let patient = {  
  id: 101,  
  name: "Rahul",  
  age: 25,  
  disease: "Fever",  
  admitted: true  
};  
  
console.log(patient.id);  
console.log(patient.name);  
console.log(patient.age);  
console.log(patient.disease);  
console.log(patient.admitted);
```

```
// Expected Output:
```

```
// 101
```

```
// Rahul
```

```
// 25
```

```
// Fever
```

```
// True
```

Actual Output (Node.js terminal):

```
PS D:\Set2> node task10.js
101
Rahul
25
Fever
true
```

GitHub Repository Link

https://github.com/RichardRajuChirayath/L-T_5th_sem_CIA_1